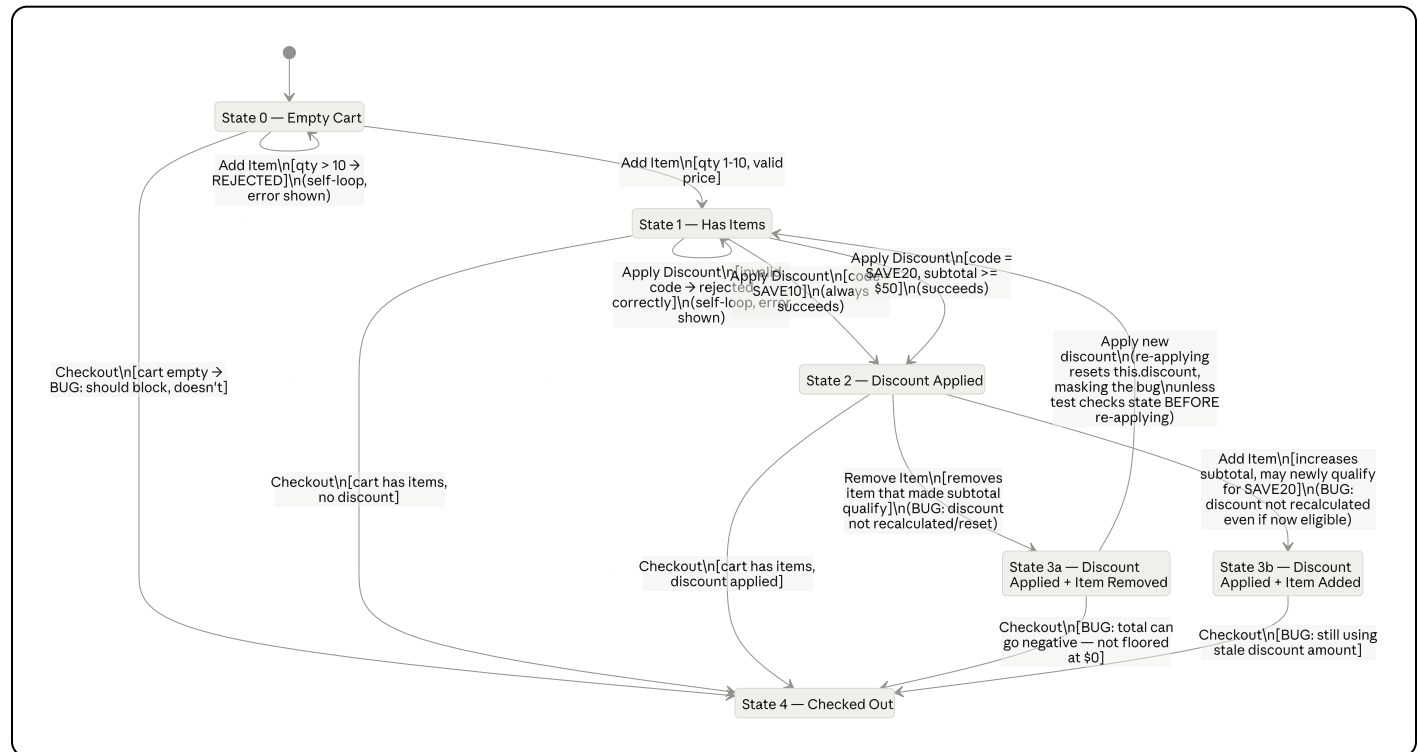


Shopping Cart — State Diagram & Test Case Derivation

1. State Diagram



Notes on the diagram

- **Self-loops matter as much as forward edges.** Every guard/validation rule in the spec should appear as either a real transition (valid input) or a self-loop (rejected input, state unchanged). If a rule has no edge, it has no test.
- **State 3 was split** into 3a (item removed after discount) and 3b (item added after discount) because they expose two *different* consequences of the same root bug (discount is calculated once and never recalculated).
- **S0 → S4 is the most important edge to not skip.** It's easy to forget "checkout" as a valid action from the empty state — but the guard that should block it is exactly what's buggy.

2. Test Case Table (derived directly from the edges above)

ID	Edge / Transition	Description	Input / Steps	Expected Result	Known bug?
TC01	S0→S1	Add valid item to empty cart	productId=1, qty=2, price=10	Item appears in list; subtotal=\$20	No
TC02	S0→S0 (self-loop)	Reject qty > 10 on empty cart	productId=1, qty=11, price=10	Error "Invalid quantity"; cart stays empty	No
TC03	S0→S0 (self-loop)	Reject qty = 0	productId=1, qty=0, price=10	Error shown; cart stays empty	No
TC04	S1→S1 (self-loop)	Reject qty > 10 with existing items	Add item A, then try qty=11 for item B	Error shown; item B not added, item A untouched	No
TC05	S1→S1 (self-loop)	Remove nonexistent item	Cart has item "1"; remove "999"	Clear error shown, no crash, cart unchanged	Yes — no existence check
TC06	S1→S2	Apply SAVE10	Add item, price=\$100; apply SAVE10	Discount = \$10; total = \$90	No
TC07	S1→S2	Apply SAVE20 when subtotal ≥ \$50	Add item, price=\$100; apply SAVE20	Discount = \$20; total = \$80	No
TC08	S1→S1 (self-loop)	Reject SAVE20 when subtotal < \$50	Add item, price=\$30; apply SAVE20	Should show error / not apply discount	Yes — missing \$50 minimum check
TC09	S1→S1 (self-loop)	Reject invalid discount code	Apply code "FAKE10"	Error "Invalid code"; no discount applied	No
TC10	Case-insensitivity	Discount code accepts lowercase	Apply "save10"	Discount applied same as "SAVE10"	No

ID	Edge / Transition	Description	Input / Steps	Expected Result	Known bug?
TC11	Only one code active	Apply SAVE10 then SAVE20	Apply SAVE10, then apply SAVE20 (subtotal $\geq$ \$50)	Only the second discount is active; not stacked	No (verify no stacking)
TC12	S0→S4	Block checkout on empty cart	Click Checkout with 0 items	Error shown; checkout blocked	Yes — check is missing entirely
TC13	S1→S4	Checkout succeeds with items, no discount	Add item, checkout	Success message; total = subtotal	No
TC14	S2→S4	Checkout succeeds with items + discount	Add item, apply SAVE10, checkout	Success; total = subtotal – discount	No
TC15	S2→S3a→S4	Total floored at \$0 after discount outlives removed item	Add item (\$100), apply SAVE20 (\$20 off), remove the item, checkout	Total = \$0 (not – \$20)	Yes — total not floored at 0
TC16	S2→S3b	Discount not recalculated when subtotal grows	Add item (\$30, doesn't qualify for SAVE20 — use SAVE10 instead), then add another item pushing subtotal $\geq$ \$50	Discount is still based on the original (smaller) subtotal; doesn't retroactively adjust	Yes — stale discount, same root cause as TC15
TC17	Boundary	Subtotal exactly \$50 qualifies for SAVE20	Add item, price=\$50; apply SAVE20	Discount = \$10 (20% of 50); succeeds	No — boundary check
TC18	Boundary	Subtotal \$49.99 does not qualify	Add item, price=49.99; apply SAVE20	Rejected	Yes (same bug as TC08, boundary-

ID	Edge / Transition	Description	Input / Steps	Expected Result	Known bug?
					focused variant)
TC19	Boundary	Quantity exactly 10 is allowed	productId=1, qty=10, price=5	Item added successfully	No
TC20	Boundary	Quantity exactly 11 is rejected	productId=1, qty=11, price=5	Error shown	No (duplicate confidence check vs TC02)

Mapping bugs → test cases (for your own tracking)

1. **Remove nonexistent item not guarded** → TC05
2. **SAVE20 missing \$50 minimum** → TC08, TC18
3. **Total not floored at \$0** → TC15
4. **Checkout allowed on empty cart** → TC12
5. *(Same root cause as #3, different trigger)* **Discount never recalculated after cart changes** → TC16

3. How to use this later

- Pick any edge in the diagram you haven't automated yet and write the Cypress test for it directly from its table row — steps and expected result are already spelled out.
- If you extend the app (e.g., add a "clear cart" button, multiple discount stacking, per-item removal quantity), redraw the diagram first, add the new edges/self-loops, *then* derive new test case rows the same way — don't add test cases without first checking whether they represent a new edge, an existing one, or a duplicate.